1 write a nodejs program to perform read,write,and other operations on a file

Write.js

```
const fs = require('fs');
fs.writeFile('demo.txt', 'Hello NodeJS\n', (err) => {
if (err) throw err;
console.log("File created and data written.");
});
```

read.js

```
const fs = require('fs');
fs.readFile('demo.txt', 'utf8', (err, data) => {
if (err) throw err;
console.log("File content:");
console.log(data);
});
```

append.js

```
const fs = require('fs');
fs.appendFile('demo.txt', 'Appending new line\n', (err) => {
if (err) throw err;
console.log("Data appended successfully.");
});
```

rename.js

```
const fs = require('fs');
fs.rename('demo.txt', 'newdemo.txt', (err) => {
if (err) throw err;
console.log("File renamed successfully.");
});
```

delete.js

```
const fs = require('fs');fs.unlink('newdemo.txt', (err) => {
if (err) throw err;
console.log("File deleted successfully.");
});
```

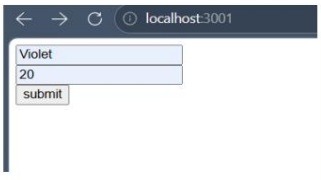
```
PS D:\Deepthi\sem6\fsd\int1> node write.js
File created and data written.
PS D:\Deepthi\sem6\fsd\int1> node read.js
File content:
Hello NodeJS
PS D:\Deepthi\sem6\fsd\int1> node append.js
Data appended successfully.
PS D:\Deepthi\sem6\fsd\int1> node read.js
File content:
```

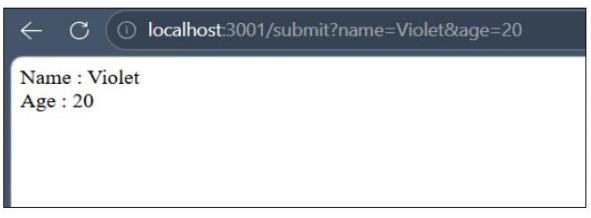
```
Hello NodeJS
Appending new line
PS D:\Deepthi\sem6\fsd\int1> node rename.js
File renamed successfully.
PS D:\Deepthi\sem6\fsd\int1> node delete.js
File deleted successfully.
PS D:\Deepthi\sem6\fsd\int1>
```

2. write a nodejs program to read form from query string and generate response using nodejs

```
const http=require('http');
const url=require('url');
const server=http.createServer((req,res)=>{
const purl=url.parse(req.url,true);
const path=purl.pathname;
if(path=='/'){
res.writeHead(200,{'Content-Type':'text/html'});
res.write(`
<form action="/submit" method="GET">
<input type="text" name="name" placeholder="Name"><br>
<input type="text" name="age" placeholder="Age"><br>
<input type="submit" value="submit">
</form>`);
res.end();
}
else if(path=='/submit'){
const name=purl.query.name;
const age=purl.query.age;
res.writeHead(200,{'Content-Type':'text/html'});
res.write("Name : "+name+"<br>");
res.write("Age : "+age);
res.end();
}
});
server.listen(3001);
```

Output:





3. write a nodejs program to demonstrate events and call back functions

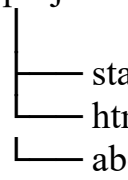
```
const events = require('events');
const eventEmitter = new events.EventEmitter();
function greet(name) {
console.log("Hello " + name);
}
function notify() {
console.log("Notification received!");
}
eventEmitter.on('welcome', greet);
eventEmitter.on('welcome', notify);
eventEmitter.emit('welcome', "Deepthi");
```

Hello Deepthi
Notification received!

4. Write a Node JS program to demonstrate accessing static files using http webserver and client.

```
const http=require('http');
const fs=require('fs');
const url=require('url');
const ROOT="html";const server=http.createServer((req,res)=>{
const path=url.parse(req.url).pathname;
fs.readFile(ROOT+path,(err,data)=>{
if(err){
res.writeHead(404);
res.write("File not found");
}else{
res.writeHead(200);
res.write(data);
}
})
});
server.listen(3002);
```

project/



localhost:3002/abc.html

5.Write a NodeJs program to Implement a program with basic commands (CRUD) on databases and collections using MongoDB.

• mongosh
• show dbs
• use MYDB1
• db.createCollection("students")
• show collections
• db.students.insertOne({rollno:501,name:"CSE3"})
• db.students.insertOne({rollno:502,name:"CSE2"})
• db.students.find().pretty()
• db.students.updateOne({rollno:502},{$set:{name:"CSE3"}})
• db.students.find().pretty()
• db.students.deleteOne({rollno:501})
• db.students.find().pretty()
• db.students.drop()
• db.dropDatabase()
• show dbs

6 Implement CRUD operations on the given dataset using mongodb and nodejs

```
//crud op
const {MongoClient} =require('mongodb');
const client=new MongoClient('mongodb://localhost:27017');
client.connect()
    .then(()=>{
        console.log("Connected successfully");
    })
    .catch(error=>console.log('Failed to connect!',error));

//insert
client.db('students').collection('cse1').insertOne({
    name:'Sneha',
    Roll:'597'
})
.then((res)=>{
    console.log(res);
})
.catch((err)=>{
    console.log(err);
});

//insert many
```

```
client.db('students').collection('cse1').insertMany([
    {name:'varshi',Roll:'536'},
    {name:'adi',Roll:'572'},
    {name:'chinmayi',Roll:'588'}
])
.then((res)=>{
    console.log(res);
})
.catch((err)=>{
    console.log(err);
});

//update
client.db('students').collection('cse1').updateOne({name:'Sneha'},{$set:{R
oll:599}})
.then((res)=>{
    console.log(res);
})
.catch((err)=>{
    console.log(err);
});

//Delete
client.db('students').collection('cse1').deleteOne({name:'Sneha'})
.then((res)=>{
    console.log(res);
})
.catch((err)=>console.log(err));

//findOne
client.db('students').collection('cse1').findOne({name:'Sneha'})
.then((res)=>{
    console.log(res);
    client.close();
})
.catch((err)=>{
    console.log(err);
})
```

7.**Perform Count, Limit, Sort, and Skip operations on the given collections**
using MongoDB.

• mongosh

• use MYDB1•
db.students.insertMany([
{rollno:501,name:"CSE3"},
{rollno:502,name:"CSE2"},
{rollno:503,name:"ECE1"},
{rollno:504,name:"MECH"},
{rollno:505,name:"EEE"}
])
• db.students.countDocuments()
• db.students.countDocuments({name:"CSE3"})
• db.students.find().limit(2)
• db.students.find().sort({rollno:1})
• db.students.find().sort({rollno:-1})
• db.students.find().skip(2)
• db.students.find().skip(2).limit(2)
• db.students.find().sort({rollno:-1}).limit(3)

8. Program to demonstarte Applying Route Parameters in Express JS

```
const express = require('express');
const app = express();
app.get('/student/:name', (req, res) => {
res.send("Student Name: " + req.params.name);
});
app.listen(3000, () => {
console.log("Server Running");
});
```

9.To demonstrate data binding (string interpolation, property binding & class binding) in Angular.

```
//app.ts
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import {FormsModule} from '@angular/forms';
@Component({
  selector: 'app-root',
  imports: [RouterOutlet,FormsModule],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
```

```
  protected readonly title = signal('oneway');
  title1='oneway';
  isdisabled:boolean=true;
  isActive:boolean=true;


//app.html
<h1> String interpolation</h1>
<p>{{title1+title}}</p>
<h1>Property binding</h1>
Name:<input type="text">
<button [disabled]="isdisabled">Submit</button>
<h1>Class binding</h1>
<h1 [class.active]="isActive">Welcome to my world</h1>

//app.css
.active{
    color:red;
}
```

10.To demonstrate data binding using events (event binding) in Angular.

```
/app.ts
import {Component,signal} from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { FormsModule } from '@angular/forms';
@Component({
  selector:'app-root',
  imports:[RouterOutlet,FormsModule],
  templateUrl:'./app.html',
  styleUrl:'./app.css'
})
export class App{
  count=0;
  name:any='latha';
  increment()
  {
    this.count++;
  }
  decrement()
  {
    this.count--;
```

```
  }
  changeName(e:any){
    this.name=e.target.value;
  }
}
```

//app.html

```
<h1>Event Binding</h1>
<h2>{{count}}</h2>
<button (click)="increment()">Increment</button>
<button (click)="decrement()">Decrement</button><br><br>
<input type="text"(input)="changeName($event)">
<h3>Your Name : {{name}}</h3>
```

11. To demonstrate two way data binding in Angular.

/app.ts

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import {FormsModule} from '@angular/forms';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet,FormsModule],
  templateUrl: './app.html',
  styleUrl: './app.css'
})
export class App {
  protected readonly title = signal('two-way');
  text='';
}
```

//app.html

```
<h1>TWO WAY BINDING</h1>
<h1>Enter text:</h1>
<input [(ngModel)]="text">
<h2>you entered :{{text}}</h2>
```

12. To demonstrate functional and class components in React.

App.jsx

```
import React from 'react';
import CBC from './CBC';
import FBC from './FBC';
function Greeting(){
 return(
   <div>
     <h1>Hello World</h1>
     <CBC></CBC>
     <FBC></FBC>
     <h2>CSE-2 Snehalatha</h2>
   </div>
 )
}
export default Greeting
```

```
//CBC
import React ,{Component} from 'react';
class CBC extends Component{
   render(){
      return(
         <div>
            <h1>CLASS COMPONENT</h1>
         </div>
      )
   }
}
export default CBC
```

```
//FBC
import React from 'react';
function FBC(){
   return(
      <div>FUNCTIONAL COMPONENT</div>
   )
}
export default FBC
```

13.To demonstrate data (state) and data flow (props) in React.

```
import React,{Component} from 'react';
class App extends Component{
```

```
  constructor(){
    super();
    this.state={
      count:0
    }
  }
    increment=()=>{
      this.setState({count:this.state.count+1});
    }
    render(){
      return(
      <div>
        <h2>{this.state.count}</h2>
        <button onClick ={this.increment}>INCREMENT</button>
      </div>
      );
    }
  }
export default App
```

14.Develop a form with a field an email address and validate it using React.

```
import React, { useState } from 'react';

function App() {
  const [email, setEmail] = useState("");

  const validate = () => {
    if (email.includes("@")) {
      alert("Valid Email");
    } else {
      alert("Invalid Email");
    }
  };

  return (
    <div>
      <h2>Email Validation</h2>

      <input
        type="text"
        placeholder="Enter Email"
        onChange={(e) => setEmail(e.target.value)}
```

```
      />

      <button onClick={validate}>Check</button>
    </div>
  );
}

export default App;
```

1. Node.js File Operations

Description:
Develop a Node.js program to perform file handling operations such as creating, reading, writing, appending, renaming, and deleting files using the File System (fs) module.

---

2. Reading Form Data Using Query String

Description:
Create a Node.js application that accepts form input through query string parameters and generates a dynamic response based on the received data.

---

3. Events and Callback Functions in Node.js

Description:
Implement a Node.js program to demonstrate the use of EventEmitter for handling events and callback functions for asynchronous execution.

---

4. Accessing Static Files Using HTTP Server

Description:
Develop a Node.js HTTP server that serves static files such as HTML, CSS, images, and JavaScript files to clients through web requests.

---

5. MongoDB CRUD Operations

Description:
Perform basic database and collection operations in MongoDB, including creating databases, creating collections, inserting, reading, updating, and deleting documents.

---

6. MongoDB CRUD Operations Using Node.js

Description:
Build a Node.js application connected to MongoDB to perform Create, Read, Update, and Delete (CRUD) operations on a given dataset.

---

7. MongoDB Count, Limit, Sort, and Skip Operations

Description:
Implement MongoDB queries to count documents, limit the number of results, sort records in ascending or descending order, and skip specific documents.

---

8. Route Parameters in Express.js

Description:
Create an Express.js application that demonstrates the use of route parameters to pass and retrieve dynamic values through URLs.

---

9. Angular Data Binding (String Interpolation, Property Binding, Class Binding)

Description:
Develop an Angular application to demonstrate different types of data binding, including string interpolation, property binding, and class binding.

---

10. Event Binding in Angular

Description:
Create an Angular application that handles user interactions such as button clicks and keyboard events using event binding.

---

11. Two-Way Data Binding in Angular

Description:
Implement two-way data binding in Angular using the ngModel directive to synchronize data between the component and the view.

---

12. Functional and Class Components in React

Description:
Develop a React application demonstrating both Functional Components and Class Components and their rendering process.

---

13. State and Props in React

Description:
Create a React application to demonstrate state management and data flow between parent and child components using props.

---

14. Email Validation Form in React

Description:
Develop a React form containing an email input field and implement validation logic to verify the correctness of the entered email address.